Fractions and Floating Point

#A

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

Announcements

- Don’t forget the points for discussing your project solution with your TA are expiring

Fractional Binary

Fractional numbers

- Up to this point we have been working with integer numbers.
  - Unsigned and signed!

2019

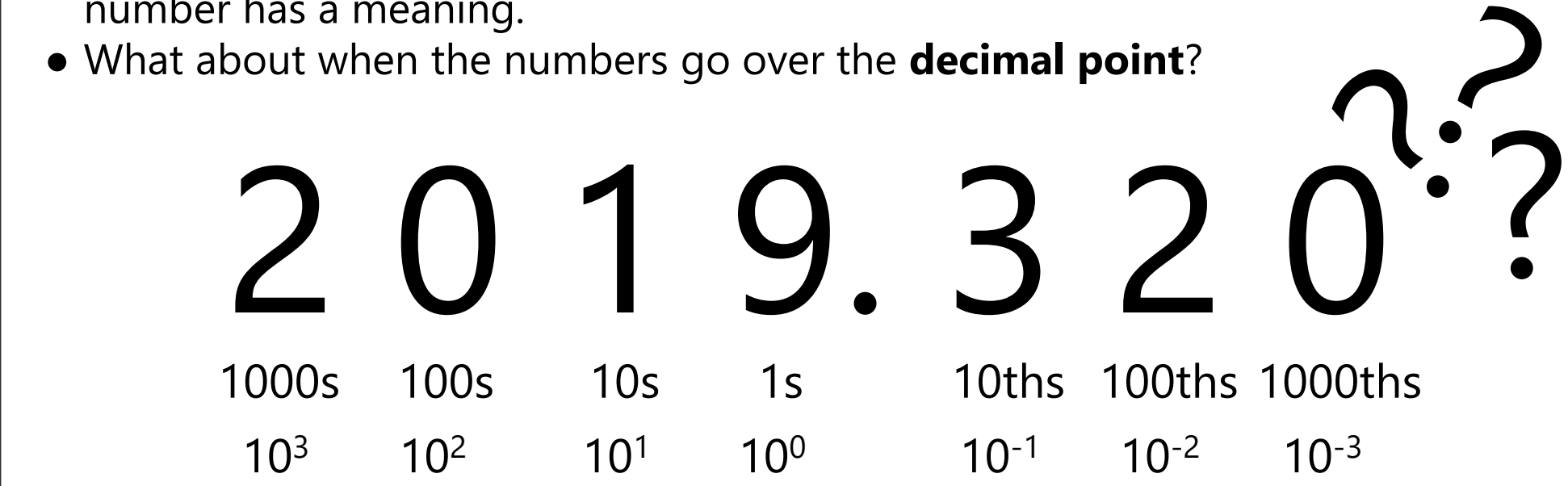
- However, Real world numbers are… Real numbers. Like so:

2019.320

- That create new challenges!
  - Let's start by taking a look at them.

Just a fraction of a number

- The numbers we use are written **positionally**: the position of a digit within the number has a meaning.
- What about when the numbers go over the **decimal point**?



A fraction of a bit?

- Binary is the same!
- Just replace 10s with 2s.

0	1	1	0	.	1	1	0	1
2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}
8s	4s	2s	1s		2ths?	4ths	8ths	16ths

To convert into decimal, just add stuff

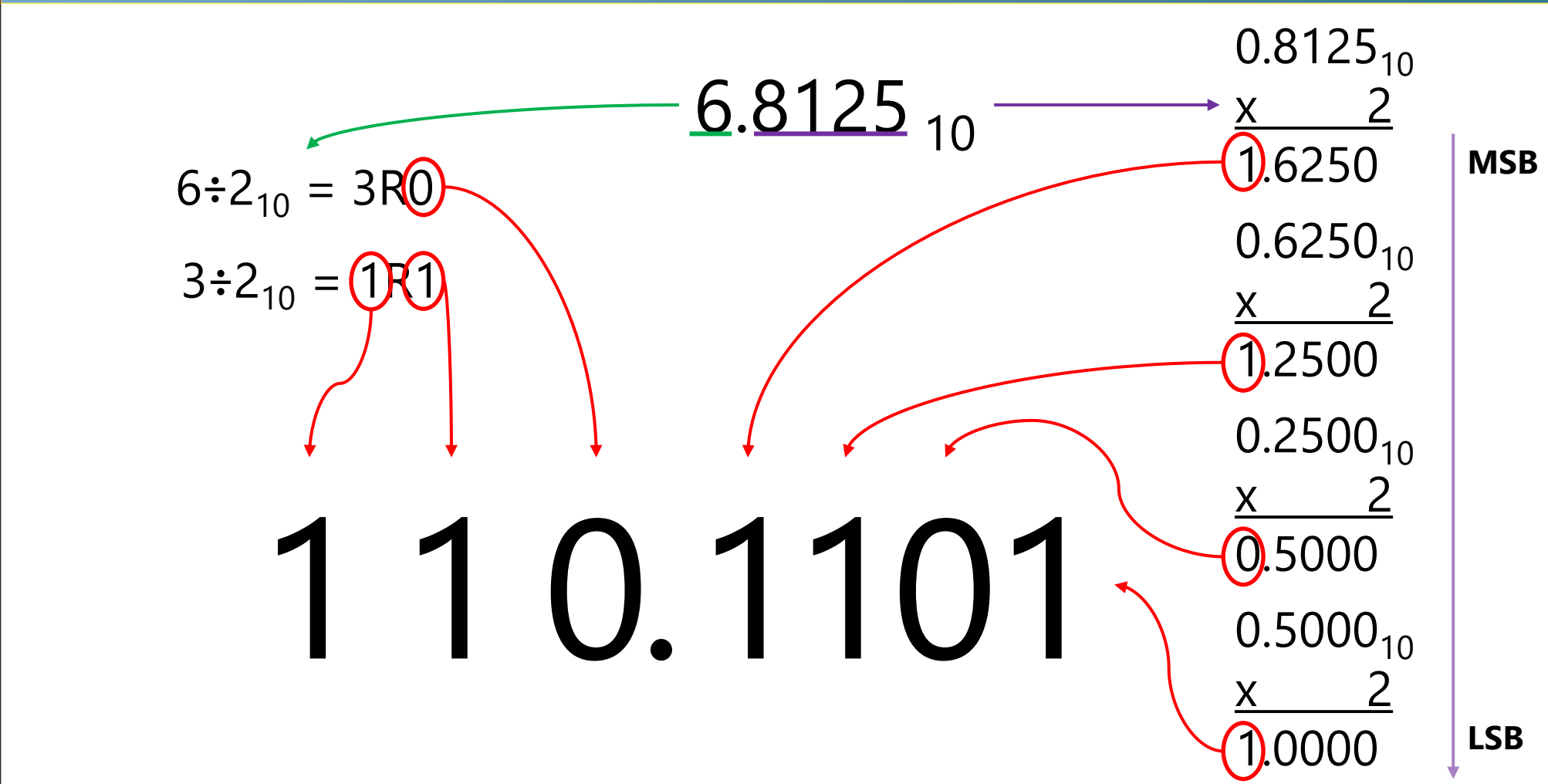
$$0\ 1\ 1\ 0.1\ 1\ 0\ 1 =$$

$$2^3\quad 2^2\quad 2^1\quad 2^0\quad 2^{-1}\quad 2^{-2}\quad 2^{-3}\quad 2^{-4}$$

$0 \times 8 +$
$1 \times 4 +$
$1 \times 2 +$
$0 \times 1 +$
$1 \times .5 +$
$1 \times .25 +$
$0 \times .125 +$
$1 \times .0625$

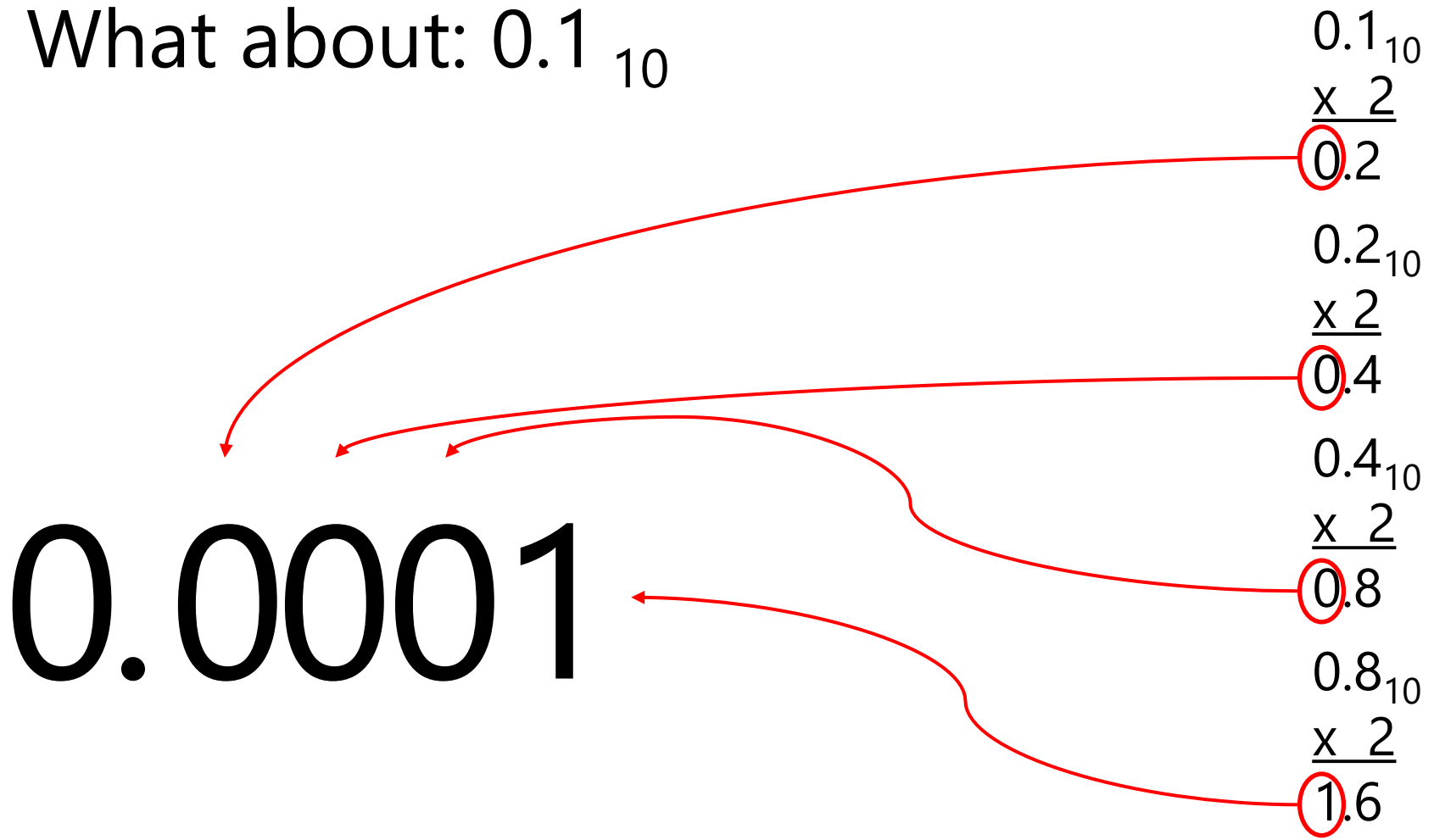
$$= 6.8125_{10}$$

From decimal to binary? Tricky?



So, it's easy right? Well…

What about: 0.1_{10}

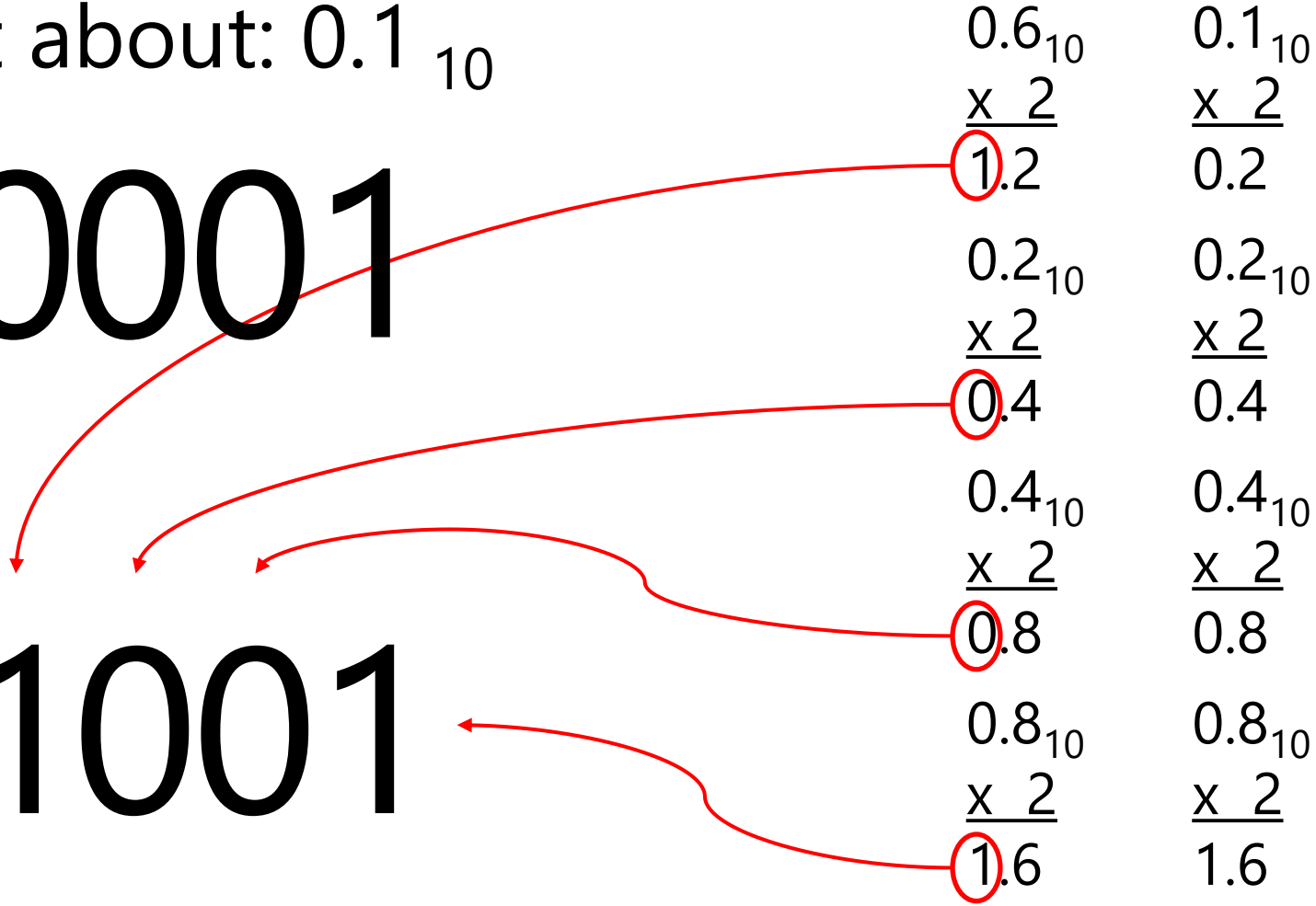


So, it's easy right? Well……

What about: 0.1_{10}

0.0001

1001



So, it's easy right? Well.....

What about: 0.1_{10}

$0.0001\ 1001\ 1001\ \ldots$

0.6_{10}	0.6_{10}	0.1_{10}
x 2	x 2	x 2
1.2	1.2	0.2
0.2_{10}	0.2_{10}	0.2_{10}
x 2	x 2	x 2
0.4	0.4	0.4
0.4_{10}	0.4_{10}	0.4_{10}
x 2	x 2	x 2
0.8	0.8	0.8
0.8_{10}	0.8_{10}	0.8_{10}
x 2	x 2	x 2
1.6	1.6	1.6

How much is it worth?

- Well, it depends on where you stop!

$0.0001_2 = 0.0625$

$0.00011001_2 = 0.0976\ldots$

$0.000110011001_2 = 0.0998\ldots$

Limited space!

- How much should we store?
  - We have 32-bit registers, so 32-bits?
    - Let's say we do!

- How many bits are used to store the integer part?
- How many bits are used to store the fractional part?



- What are the tradeoffs?

A rising tide

- Maybe half-and-half? 16.16 number looks like this:

0011 0000 0101 1010.1000 0000 1111 1111

binary point

the largest (signed) value we can represent is +32767.999

the smallest fraction we can represent is 1/65536

What if we place the binary point to the left…

0011.0000 0101 1010 1000 0000 1111 1111

…we can get much higher **accuracy** near 0…

…but if we place the binary point to the right…

0011 0000 0101 1010 1000 0000.1111 1111

…then we trade off accuracy for **range** further away from 0.

Mind the point

- In this representation we assume that the lowest *n* digits are the decimal places.

$12.34	1234
+$10.81	+1081
$23.15	2315

this is called **fixed-point representation**

And it's a **bitfield :D**

Move the point

- What if we could **float** the point around?
  - Enter scientific notation: The number **-0.0039** can be represented:

$$-0.39 \times 10^{-2}$$

$$-3.9 \times 10^{-3}$$

- These are both representing the **same number**, but we need to move the decimal point according to the power of ten represented.

- The bottom example is in **normalized scientific notation.**
  - There is only **one non-zero** digit to the left of the point

- Because the decimal point can be moved, we call this representation

Floating point

Floating-point number representation

This could be a whole unit itself…

- floating-point arithmetic is COMPLEX STUFF

- However…
  - it's good to have an understanding of *why* limitations exist
  - it's good to have an *appreciation* of how complex this is… and how much better things are now than they were in the 1970s and 1980s
  - It's good to know things do not behave as expected when using float and double!

Binary numbers using IEEE 754

- est'd 1985, updated as recently as 2008
- standard for floating-point representation and arithmetic that **virtually every CPU** now uses
- floating-point representation is based around **scientific notation**

$$1348 = +1.348 \times 10^{+3}$$
$$-0.0039 = -3.9 \times 10^{-3}$$
$$-1440000 = -1.44 \times 10^{+6}$$



Binary Scientific Notation

- scientific notation works equally well in any other base!
  - (below uses base-10 exponents for clarity)

$$+1001\ 0101 = +1.001\ 0101 \times 2^{+7}$$
$$-0.001\ 010 = -1.010 \times 2^{-3}$$
$$-1001\ 0000\ 0000\ 0000 = -1.001 \times 2^{+15}$$



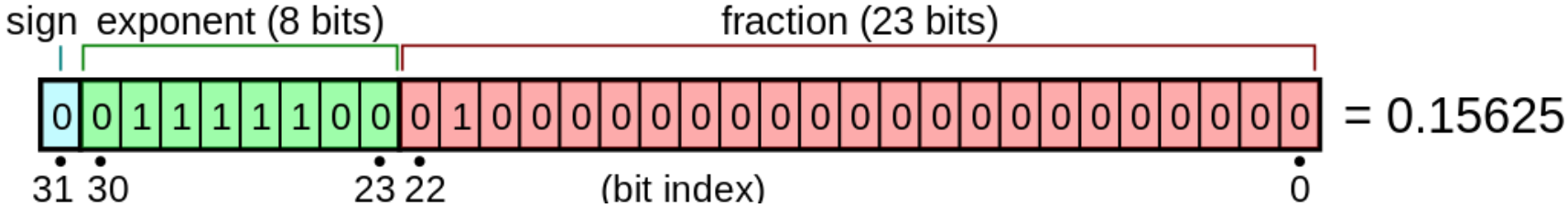
what do you notice about the digit before the **binary** point using normalized representation?

$$(-1)^s \times 1.f \times 2^{exp}$$

s – sign
f – fraction
1.f – significand
exp – exponent

IEEE 754 Single-precision

- Known as **float** in C/C++/Java etc., 32-bit float format
- 1 bit for sign, 8 bits for the exponent, 23 bits for the *fraction*

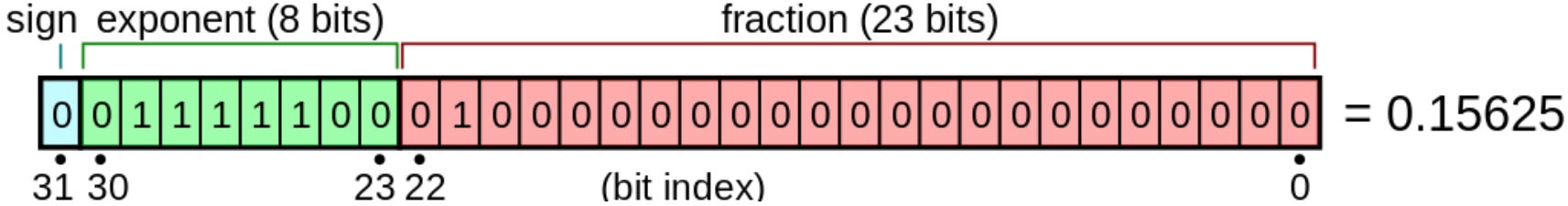


  - Tradeoff:
    - More **accuracy** ➔ More fraction bits
    - More **range** ➔ More exponent bits
  - Every design has tradeoffs ¯_(ツ)_/¯

illustration from user Stannered on Wikimedia Commons

IEEE 754 Single-precision

- Known as **float** in C/C++/Java etc., 32-bit float format
- 1 bit for sign, 8 bits for the exponent, 23 bits for the *fraction*



  - The fraction field *only stores the digits after the binary point*
  - ***The 1 before the binary point is implicit!***
    - This is called normalized representation
    - In effect this gives us a *24-bit* significand
  - The significand of floating-point numbers is **in sign-magnitude!**
    - Do you remember the downside(s)?

illustration from user Stannered on Wikimedia Commons

The exponent field

- the exponent field is 8 bits, and can hold positive or negative exponents, but... it doesn't use S-M, 1's, or 2's complement.
- it uses something called **biased notation.**
  - biased representation = exponent + *bias constant*
  - **single-precision** floats use a bias constant of **127**

```
exp + 127 => Biased
```

```
-127 + 127 => 0
 -10 + 127 => 117
  34 + 127 => 161
```

  - the exponent can range from **-126 to +127** (1 to 254 biased)
    - 0 and 255 are reserved!
  - why'd they do this?
    - You can sort floats with integer comparisons!

Binary Scientific Notation (revisited)

- Our previous numbers are actually

$$+1.001\ 0101 \times 2^{+7}$$

bias = 127

sign = 0 (positive number!)

Biased exponent = exp + 127 = 7 + 127 = 134

= 10000110

fraction = 0010101 (ignore the "1.")

s	E	f
0	10000110	00101010000000000…000

$$(-1)^0 \times 1.001\ 0101 \times 2^{134-127}$$

Binary Scientific Notation (revisited)

- Our previous numbers are actually

bias = 127

$-1.010 \times 2^{-3} =$

sign = 1 (negative number!)

Biased exponent = exp + 127 = -3 + 127 = 124

= 01111100

fraction = 010 (ignore the "1.")

s	E	f
1	01111100	01000000000000000…000

$(-1)^1 \times 1.010 \times 2^{124-127}$

Binary Scientific Notation (revisited)

- Our previous numbers are actually

$$-1.001 \times 2^{+15} =$$

bias = 127

sign = ?

Biased exponent = ?

fraction = ?

s	E	f
?	?	?

Check it on C++ (there are online tools for this!)

```
#include <iostream>
#include<bitset>
int main() {
    // This is the number from the previous slide
    float x {-0b1001000000000000};

    // C++ does not shift floats :(
    // This is C++-whispering: it allows me to shift the bits :)
    int num {*(int*)&x};

    // Extract the fields!
    std::bitset <1> sign = (num >> 31) & 0x1 ;
    std::bitset <8> biased_exp = (num >> 23) & 0xFF;
    std::bitset <23> frac = (num >> 0) & 0x7FFFFF;

    // Now let's print
    std::cout << "sign: " << sign << std::endl;
    std::cout << "biased exponent: " << biased_exp << std::endl;
    std::cout << "frac: " << frac << std::endl;
    return 0;
}
```

Try it in: https://repl.it/languages/cpp

Encoding a number as a float

You have an number, like -12.59375_{10}

1. Convert it to binary.
   Integer part: 1100_2
   Fractional part: 0.10011_2
2. Write it in scientific notation:
   $1100.10011_2 \times 2^0$
3. Normalize it:
   $1.10010011_2 \times 2^3$

0.59375_{10} x 2 = 1.18750

0.18750_{10} x 2 = 0.37500

0.37500_{10} x 2 = 0.75000

0.75000_{10} x 2 = 1.50000

0.50000_{10} x 2 = 1.00000

MSB

LSB

Encoding a number as a float

You have an number, like -12.59375_{10}

1. Convert it to binary.
   Integer part: 1100_2
   Fractional part: 0.10011_2
2. Write it in scientific notation:
   $1100.10011_2 \times 2^0$
3. Normalize it:
   $1.10010011_2 \times 2^3$
4. Calculate biased exponent
   $+3 + 127 = 130_{10} = 10000010_2$

0xC1498000

s	exponent	fraction
1	10000010	10010011000000000…000

Adding floating point numbers

$$1.11 \times 2^{0} + 1.00 \times 2^{-2}$$

- Step 1 – Make both exponents the same

$$1.11 \times 2^{0} + 0.01 \times 2^{0}$$

- Step 2 – Add the significands

$$1.11 \times 2^{0} + 0.01 \times 2^{0} = 10.00 \times 2^{0}$$

- Step 3 – Normalize the result

$$10.00 \times 2^{0} = 1.000 \times 2^{1}$$

Multiply floating point numbers

$$1.11 \times 2^{0} \times 1.01 \times 2^{-2}$$

- Step 1 – Add the exponents

$$0 + (-2) = [0+127]+[-2+127] =$$
$$[127] + [125] - 127 = [125] = -2$$

- Step 2 – Multiply the significands

$$1.11 \times 1.01 = 10.0011$$

- Step 3 – Normalize the result

$$10.0011 \times 2^{-2} = 1.00011 \times 2^{-1}$$

Divide floating point numbers

$$1.001 \times 2^{0} / 1.1 \times 2^{-2}$$

- Step 1 – Subtract the exponents

$$0 - (-2) = [0+127]-[-2+127] =$$
$$[127] - [125] + 127 = [129] = 2$$

- Step 2 – Divide the significands

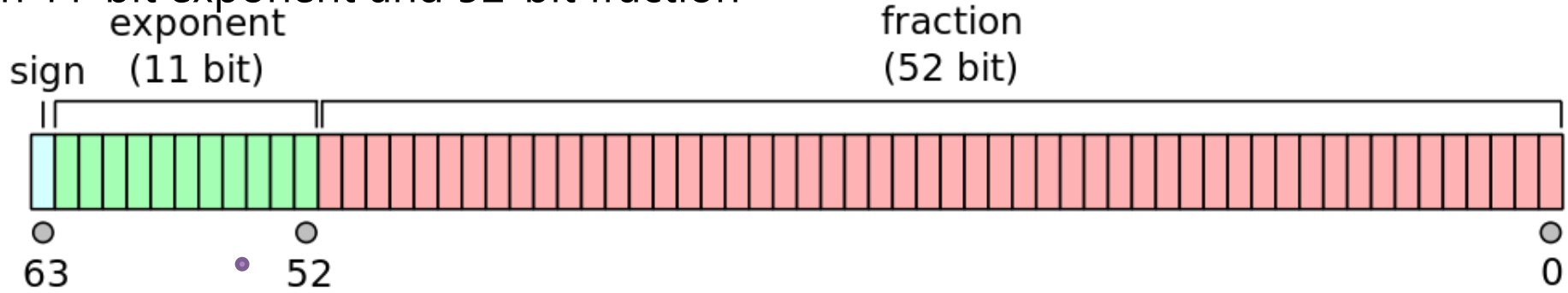
$$1.001 / 1.1 = 0.11$$

- Step 3 – Normalize the result

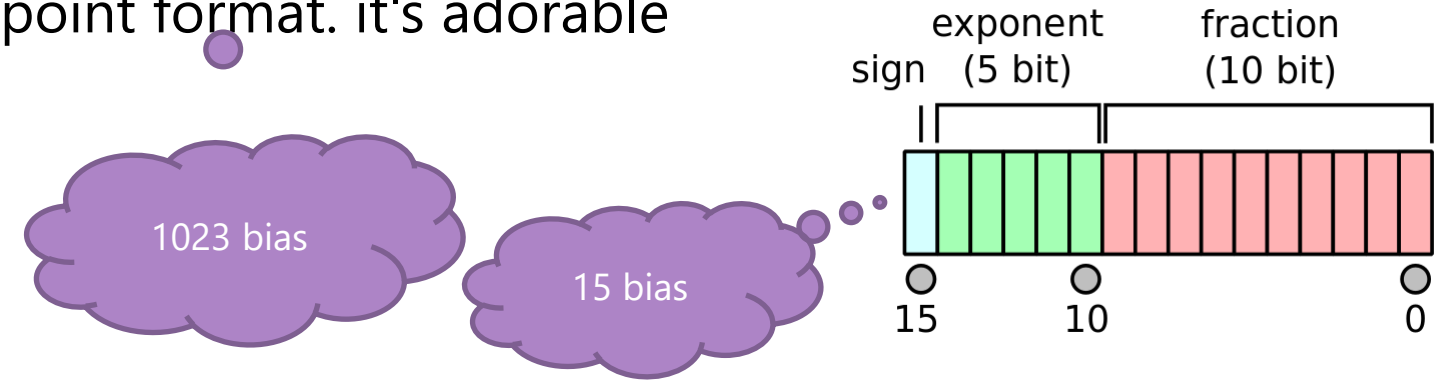
$$0.11 \times 2^{2} = 1.1 \times 2^{1}$$

Other formats

- the most common other format is **double-precision** (C/C++/Java **double**), which uses an 11-bit exponent and 52-bit fraction



  - GPUs have driven the creation of a **half-precision** 16-bit floating-point format. it's adorable



both illustrations from user Codekaizen on Wikimedia Commons

Special cases

- IEEE 754 can represent data outside of the norm.
  - Zero! How do you do that with normalized numbers?
  - +/- Infinity
  - NaN (Not a number). E.g. when you divide zero by zero.
  - Other denormalized number: Squeeze the most out of our bits!
    - E.g.: $0.00000000000000000000001 \times 2^{-127}$

Single precision		Double precision		Meaning
Exponent	**Fraction**	**Exponent**	**Fraction**	
0	0	0	0	0
0	!=0	0	!=0	Number is denormalized
255	0	2047	0	Infinity (sign-bit defines + or -)
255	!=0	2047	!=0	NaN (Not a Number)

Check out this cool thing in MARS

- go to Tools > Floating Point Representation
- Try it out!

